DDD, Hexagonal Architecture & Dependency Injection Cheatsheet

Lessons: 31 — DDD (tactical) · 32 — Hexagonal / Ports & Adapters · 33 — Dependency Injection · 45 — Polymorphic Relations

Examples: 31 · 32 · 33

Covers: entities, value objects, aggregates, ports/adapters, wiring, type registries

Legend: `[*]` = concept or tool the lessons have not covered yet

DDD BUILDING BLOCKS

```
Entity                      has an IDENTITY that outlives its values
                            two Users with the same ID are the same user
Value Object                defined ONLY by its values; immutable; compared by ==
                            Money, Email, DateRange — no ID, replace don't mutate
Aggregate                   a cluster of objects with one ROOT and one invariant
Aggregate Root              the only entry point; outsiders hold a reference to it
Repository                  collection-like access to aggregate ROOTS only
Domain Service              logic belonging to no single entity (transfer, pricing)
Domain Event                something that happened, past tense: OrderPlaced
Factory                     builds a valid aggregate from raw input
Bounded Context          [*] one model, one meaning, one team; "User" differs by it
Ubiquitous Language         the code uses the words the business uses
```

VALUE OBJECTS IN GO

```
type Money struct { cents int64; currency string }    unexported fields
func NewMoney(c int64, cur string) (Money, error)     validate at construction
func (m Money) Add(o Money) (Money, error)            return a NEW value
value receivers             a VO never mutates
comparable with ==          if every field is comparable
type Email string           the lightweight form + a NewEmail validator
(if it can be invalid after construction, it isn't a value object)
```

AGGREGATES & INVARIANTS

```
type Order struct {         the root
  id OrderID
  items []Item              children — no repository of their own
  status Status
}
func (o *Order) AddItem(i Item) error {
  if o.status != Draft { return ErrOrderLocked }     THE INVARIANT
  o.items = append(o.items, i)
  return nil
}
no setters                  every change goes through a method that can refuse
one transaction per aggregate    save the whole root, atomically
reference other aggregates BY ID    never hold a *Customer inside an Order
keep them small             a big aggregate is a big lock
(if a rule spans two aggregates, it belongs in a domain service or an event)
```

DOMAIN EVENTS

```
type OrderPlaced struct { OrderID string; At time.Time }    past tense, immutable
o.events = append(o.events, OrderPlaced{...})      recorded by the aggregate
func (o *Order) PullEvents() []Event                drained by the service layer
published AFTER the transaction commits             or via the outbox (lesson 34)
inside the same context      keep them synchronous; cross-context goes on the bus
(events are facts — a handler that fails does not un-happen the event)
```

THE FRAMEWORK-FREE DOMAIN

```
internal/domain imports      only the stdlib and other domain packages
NOT net/http                 no status codes in the domain
NOT database/sql             no sql.NullString in an entity
NOT your JSON tags           the API shape is a DTO's problem, not the domain's
no ORM base struct           gorm.Model in an entity is the classic leak
the test proves it           domain tests need no database and no server
Anti-Corruption Layer        translate an external model into yours at the edge
```

PORTS & ADAPTERS (hexagonal)

```
driving port (inbound)       what the outside can ASK the app to do
                             = the service/use-case interface; HTTP and gRPC call it
driven port (outbound)       what the app NEEDS from the outside
                             = UserRepo, Mailer, PaymentGateway — declared by the core
adapter                      a concrete implementation of a port
                             postgres.UserRepo, smtp.Mailer, http.Handler
the dependency rule          adapters depend on the core; the core depends on nothing
inversion                    the core DEFINES the interface, infra implements it
in-memory adapters           the whole point: the core is testable with maps
layered (25) vs hexagonal    same rule, different picture — infra points inward
```

THE SHAPE IN GO

```
internal/domain/             entities, value objects, domain errors
internal/app/ (or service/)  use cases + the PORT interfaces they need
internal/adapter/postgres/   driven adapter: implements app.UserRepo
internal/adapter/http/       driving adapter: calls app.UserService
cmd/api/main.go              the composition root, wiring adapters into the core

type UserRepo interface {                  declared in app/, not in postgres/
  ByID(ctx context.Context, id string) (*domain.User, error)
  Save(ctx context.Context, u *domain.User) error
}
type UserService struct { repo UserRepo }  depends on the INTERFACE
func NewUserService(r UserRepo) *UserService
```

DEPENDENCY INJECTION

```
composition root             ONE place (main) where concrete types are chosen
constructor injection        pass dependencies in; store them on the struct
  cfg := config.Load()
  db := postgres.Connect(cfg)
  repo := postgres.NewUserRepo(db)
  svc := app.NewUserService(repo, clock, mailer)
  h := httpadapter.NewHandler(svc)
interface at the consumer     the service declares what it needs, narrowly
inject the clock              func() time.Time — the cheapest testability win
inject randomness / uuid      same reason
manual DI scales further than people expect
google/wire              [*] compile-time codegen: a provider set, no reflection
uber/fx                  [*] runtime container with lifecycle hooks
kill globals                 var db *sql.DB at package scope is the anti-pattern
kill service locators        Container.Get("userRepo") is a global with ceremony
(the test is: can you build the service twice, with different dependencies?)
```

POLYMORPHIC RELATIONS & TYPE REGISTRIES

```
the problem                  one child table (comments, attachments, audit) for
                             MANY parents (orders, cases, users)
the shape                    (subject_type, subject_id) instead of one FK per parent
  comments(id, subject_type, subject_id, body)
the trade-off                no foreign key — the database cannot enforce it
                             so YOU validate the pair on write
the registry                 one interface + map[SubjectType]Repo
  type SubjectRepo interface { Exists(ctx, id) (bool, error); CanView(ctx, u, id) bool }
  var registry = map[SubjectType]SubjectRepo{}
  func Register(t SubjectType, r SubjectRepo) { registry[t] = r }
why                          generic operations (comment, audit, attach) never
                             switch on type; adding a subject = one Register call
authorization                the registry answers "can this user see that subject?"
index                        (subject_type, subject_id) together, always
```

TRAPS & MEMORIZE

```
anemic domain model          structs with only getters/setters; the logic leaked out
entity with public setters   the invariant can be broken from anywhere
aggregate that holds objects  hold IDs across aggregate boundaries
one giant aggregate          every write contends on the same root
domain importing infra       the moment it does, the layering is decoration
interface in the adapter pkg  wrong side — the consumer owns the interface
one interface per struct     ceremony; add the interface when there are 2 uses
DI framework on day one      manual wiring is 20 lines and needs no docs
globals for db/logger/clock  untestable, and parallel tests collide
time.Now() in the domain     the test can never assert on it
polymorphic FK without a check  orphan rows nobody notices for a year
switch on subject_type       the registry exists so you never write that switch
```